

Portfolio Milestone Module 6: First-Order Logic for CubeSat Telemetry Diagnosis

Alexander Ricciardi

Colorado State University Global

CSC510: Foundations of Artificial Intelligence

Professor: Dr. Isaac Gang

August 30, 2026

Portfolio Milestone Module 6: First-Order Logic for CubeSat Telemetry Diagnosis

The proposed CubeSat telemetry anomaly-detection program from “Portfolio Milestone Module 2: Use-Case Scenario Proposal” (Ricciardi, 2026) will use first-order logic as the knowledge-representation and reasoning layer that connects the neural-network classifier, similarity search, expert-system rules, and A* diagnostic planner. First-order logic uses objects as its basic components, represents individual objects with constants and relationships with predicates, and uses variables, logical connectives, and quantifiers to construct more general expressions (Sharma et al., n.d.). In other words, first-order logic will not calculate an anomaly probability from numerical telemetry. Because classical first-order logic evaluates predicates as true or false, the program will keep the neural-network probability as a separate numerical value and convert the result into a logic fact only when it meets a defined threshold (e.g., an anomaly probability ≥ 0.80) or evidence rule (e.g., a critical telemetry channel exceeds its accepted range). The neural network will perform the numerical computation task, and the logic layer will convert the neural network’s anomaly classifications and other program system results into predicates that are evaluated as true or false, such as *Anomalous(segment₄₂)*, *Deviates(segment₄₂, battery_{voltage})*, and *SimilarTo(segment₄₂, case₁₁₇)*. The program will also store operator-reviewed domain knowledge as general rules. For example, the rule $\forall s \forall c [(Anomalous(s) \wedge Deviates(s, c) \wedge Critical(c)) \rightarrow RequiresCheck(s, c)]$ uses the universal quantifier $\forall$, meaning “for every” or “for all,” to state that for every telemetry segment s and every channel c , if the segment is anomalous, the channel shows a deviation, and that channel is considered important, then the program should require that channel to be checked. In contrast, the expression $\exists c [Critical(c) \wedge Deviates(segment_{42}, c)]$ uses the existential quantifier

$\exists$, meaning “there exists at least one,” to ask whether $segment_{42}$ contains at least one important telemetry channel that deviated from its accepted range.

To keep the Python implementation small and predictable, the knowledge base will use Horn clauses, which are a practical subset of first-order logic, with forward chaining; see Appendix A. The inference engine will use unification to match rule variables with facts for the selected segment, derive conclusions such as $SuspectedFault(segment_{42}, power_{subsystem})$ or $ConflictingEvidence(segment_{42})$, and record the matched facts and rule behind each conclusion. For a description of the unification concept, see Appendix A. This trace will give the operator a readable reason for the recommendation instead of presenting only the neural network’s probability. For example, if the classifier marks a segment as anomalous, the battery-voltage channel exceeds its accepted range, and similar reviewed cases involve power instability, the logic engine can infer a power-subsystem hypothesis and send *RequiresCheck* facts to the A* planner. The planner will then arrange only the relevant diagnostic checks by estimated cost. First-order logic will determine what conclusions follow from the available facts, while A* will determine an efficient order for testing those conclusions. A separate rule can infer $ManualReviewRequired(s)$ from explicit signs of conflicting or insufficient evidence, which prevents the program from turning uncertainty into a false claim of certainty. Sharda et al. (2024) explain that intelligent systems must organize acquired knowledge, validate it for consistency, and reason from it to provide solutions or recommendations; they also state that the quality of automated decisions depends on the quality of the rules. Thomason (2024) describes logic-based AI as formalizing knowledge for mechanized reasoning, and NASA’s Advanced Caution and Warning System demonstrates the practical value of combining anomaly detection, model-based fault isolation, system-effects analysis, and operator-facing diagnostic information in a space-

system setting (Colombano et al., 2013). The program will therefore test classification-linked rules against labeled OPSSAT-AD cases (Ruszczak et al., 2025), test fault-specific rules with simulated diagnostic scenarios, run targeted checks for mutually exclusive conclusions, and display the derived fault hypothesis, supporting facts, triggered rule, and recommended checks. First-order logic will not replace the operator or the numerical model. It will turn their outputs into a consistent and inspectable reasoning chain that helps the operator decide what to examine next.

References

- Colombano, S. P., Spirkovska, L., Baskaran, V., Aaseng, G., McCann, R. S., Ossenfort, J., Smith, I., Iverson, D. L., & Schwabacher, M. (2013). A system for fault management and fault consequences analysis for NASA's Deep Space Habitat. In *AIAA SPACE 2013 Conference and Exposition*. American Institute of Aeronautics and Astronautics.
<https://doi.org/10.2514/6.2013-5319>
- Ricciardi, A. (2026). *Portfolio milestone module 2: Use-case scenario proposal* [Unpublished course assignment]. Colorado State University Global.
- Ruszczak, B., Kotowski, K., Evans, D., & Nalepa, J. (2025). The OPS-SAT benchmark for detecting anomalies in satellite telemetry. *Scientific Data*, 12, Article 710.
<https://doi.org/10.1038/s41597-025-05035-3>
- Sharda, R., Delen, D., & Turban, E. (2024). *Business intelligence, analytics, data science, & AI* (5th ed.). Pearson Education.
- Sharma, N., Hug, J., Liang, J., & Zhu, H. (n.d.). *10.7 first order logic. Introduction to artificial intelligence*. University of California, Berkeley.
<https://inst.eecs.berkeley.edu/~cs188/textbook/logic/first-order-logic.html>
- Thomason, R. (2024). Logic-based artificial intelligence. In E. N. Zalta & U. Nodelman (Eds.), *The Stanford encyclopedia of philosophy* (Spring 2024 ed.). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2024/entries/logic-ai/>

Appendix A

This appendix provides additional explanations of the first-order logic notation and example predicates used in the proposed CubeSat telemetry anomaly-detection program.

Table A1

Logic Symbols

Symbol		Use in the Program
$\forall$	“For every” or “for all”	Applies a rule to every telemetry segment or channel that satisfies its conditions.
$\exists$	“There exists at least one”	Determines whether at least one object satisfies the specified conditions.
$\wedge$	AND	Requires two or more conditions to be true together.
$\rightarrow$	IF THEN	Indicates that when the conditions on the left are true, the conclusion on the right can be inferred.
s	Telemetry segment variable	Can refer to any telemetry segment.
c	Telemetry channel variable	Can refer to any telemetry channel.

Note: The table lists the first-order logic symbols and variables used.

Example Facts

Anomalous(segment₄₂)

The program has determined that telemetry segment 42 meets the defined conditions for being classified as anomalous.

Deviates(segment₄₂, battery_voltage)

The battery-voltage telemetry channel in segment 42 has deviated from its accepted range.

Critical(battery_voltage)

The battery-voltage channel has been identified in the knowledge base as a critical telemetry channel.

SimilarTo(segment₄₂, case_117)

The similarity-search component has determined that segment 42 is sufficiently similar to previously reviewed case 117.

Example Universal Rule

$$\forall s \forall c [(Anomalous(s) \wedge Deviates(s, c) \wedge Critical(c)) \rightarrow RequiresCheck(s, c)]$$

The symbols $\forall s \forall c$ mean that the rule applies to every telemetry segment s and every telemetry channel c . The rule states that if a telemetry segment is anomalous, a channel within that segment has deviated, and that channel is considered critical, then the program can infer that the channel requires further examination. For example, if the knowledge base contains:

$$Anomalous(segment_{42})$$

$$Deviates(segment_{42}, battery_{voltage})$$

$$Critical(battery_{voltage})$$

the rule allows the inference engine to derive:

$$RequiresCheck(segment_{42}, battery_{voltage})$$

This derived fact can then be passed to the A* diagnostic planner as a diagnostic check that should be considered when constructing an investigation sequence.

Example Existential Query

$$\exists c [Critical(c) \wedge Deviates(segment_{42}, c)]$$

The symbol $\exists c$ means that the program is looking for at least one telemetry channel c that satisfies the conditions inside the brackets. In this example, the program is asking whether segment 42 contains at least one critical channel that has deviated from its accepted range.

If $battery_{voltage}$ is both critical and deviating, then the query is satisfied because at least one channel meets both conditions.

Example Fault Hypothesis

A rule may combine several pieces of evidence to derive a possible fault hypothesis. For example, evidence showing that a segment is anomalous, that battery voltage has deviated, and that similar historical cases involved power instability could support the conclusion:

$$\textit{SuspectedFault}(\textit{segment}_{42}, \textit{power}_{\textit{subsystem}})$$

This conclusion does not mean that the power subsystem has been proven to have failed. It identifies a hypothesis supported by the available facts and rules that the diagnostic planner and operator can investigate further.

Example Manual-Review Conclusion

When the available evidence is conflicting or insufficient, the knowledge base can derive:

$$\textit{ManualReviewRequired}(\textit{segment}_{42})$$

This conclusion directs the program to involve the human operator rather than automatically treating an uncertain fault hypothesis as a confirmed diagnosis.

Horn Clauses

A Horn clause is a restricted form of a logical rule that contains no more than one positive conclusion. In the proposed CubeSat telemetry anomaly-detection program, Horn clauses will be used to express diagnostic rules in a simple and predictable form. For example:

$$\textit{Anomalous}(s) \wedge \textit{Deviates}(s, c) \wedge \textit{Critical}(c) \rightarrow \textit{RequiresCheck}(s, c)$$

This rule states that if a telemetry segment s is anomalous, a telemetry channel c deviates, and that channel is critical, then the program can conclude that the channel requires further examination. Using Horn clauses keeps the knowledge base easier to implement and allows the inference engine to apply rules systematically.

Unification

Unification is the process of matching variables in a general rule with specific objects found in known facts. It allows the inference engine to determine whether a rule can be applied to a particular telemetry segment or channel. For example, consider the rule:

$$Anomalous(s) \wedge Deviates(s, c) \wedge Critical(c) \rightarrow RequiresCheck(s, c)$$

and the facts:

$$Anomalous(segment_{42})$$

$$Deviates(segment_{42}, battery_{voltage})$$

$$Critical(battery_{voltage})$$

Through unification, the program matches the variable s with $segment_{42}$ and the variable c with $battery_{voltage}$. Once these variables are matched consistently across the rule, the program can derive:

$$RequiresCheck(segment_{42}, battery_{voltage})$$

Unification therefore allows a general rule written with variables to be applied to specific telemetry data.

Forward chaining is a reasoning method that begins with known facts and repeatedly applies rules to derive new facts. The process continues until no additional conclusions can be produced or until the program derives the information needed for the diagnostic task. For example, if the knowledge base contains:

$$Anomalous(segment_{42})$$

$$Deviates(segment_{42}, battery_{voltage})$$

$$Critical(battery_{voltage})$$

the inference engine can apply the Horn-clause rule and derive:

$$RequiresCheck(segment_{42}, battery_{voltage})$$

That new fact can then be used by another rule or passed to the A* diagnostic planner. In this way, forward chaining builds the reasoning process from the available telemetry evidence toward diagnostic conclusions.